Parallel SHA-256 Hashing

ريانا صقور

ملخص تنفيذي

يقدّم هذا التقرير أداة عملية لتقييم قوة كلمات المرور عبر محاكاة هجوم القوة الغاشمة (Brute-force) باستخدام تجزئةSHA-256 ، منفَّذة بنسختين متطابقتين منطقيًا — تسلسلية ومتفرعة — لإثبات تسريع حوسبي حقيقي وقابل للقياس. تغطي القياسات المرفقة متغيرين مستقلّين كما يتطلب التكليف: عدد النياسب (N = 1, 2, 4, 8, 16) وحجم حمل العمل (من 2 إلى 15.5 مليون تجزئة)، عبر ثمانية تشغيلات فعلية منفصلة، بالإضافة لدراسة تكرار (6 محاولات) لتقدير تباين القياس. تُظهر النتائج مجتمعة أن التسريع يستوي عند ~2 2.3x–xبما يطابق تمامًا عدد الأنوية الفيزيائية المتاحة (2)، وهي ظاهرة متوقعة ومُفسَّرة بدقة عبر قانون أمدال (Amdahl's Law) مطبَّقًا على عتاد محدود الموارد.

المقدمة والمشكلة

تهدف الوظيفة إلى نقل مفاهيم إدارة النياسب (Thread Management) من النظرية إلى تطبيق عملي ملموس، عبر اختيار خوارزمية قابلة للتفريع وتنفيذها بنسختين (Single-threaded / Multi-threaded) ضمن تطبيق يحل مشكلة حقيقية. المجال المختار هو التجزئة المتوازية (Parallel Hashing) ضمن مسار "المحاكاة والأمن السيبراني"، والتطبيق العملي: مدقق قوة كلمات المرور — أداة تُقدِّر زمن كسر كلمة مرور معيَّنة أمام مهاجم بقدرة حوسبية معروفة (مُقاسة فعليًا من البرنامج نفسه)، وتترجم ذلك إلى تصنيف عملي (ضعيفة/متوسطة/قوية).

المعمارية وآليات المزامنة

1. فصل الاهتمامات(Separation of Concerns)

الكود مقسوم إلى ثلاث حزم مستقلة:

- core: CandidateSpaceتمثيل فضاء المرشحات كدالة نقية فهرس↔سلسلة BruteForceHashCrackerالخوارزمية بنسختيها PasswordStrengthEvaluatorتحويل الأداء إلى تقييم عملي
- benchmark: BenchmarkRunnerحصاد أداء مستقل بحمل عمل ثابت، منفصل عمدًا عن منطق "الكسر الفعلي"
- gui: MainWindow — الحزمة الوحيدة التي تستورد الحزمتين السابقتين، محققةً فصل منطق التفرع عن واجهة الاستخدام كما يشترط التكليف.

2. تقسيم العمل(Data-Parallel Decomposition)

CandidateSpace.candidateAt(index)دالة نقية بلا حالة مشتركة تُحوِّل أي فهرس صحيح ضمن [0, total)إلى سلسلة المرشح المقابلة (نظام عد بأساس = حجم مجموعة المحارف). هذا يسمح بتقسيم المجال الكلي

إلى نطاقات متجاورة متساوية تقريبًا، كل نطاق يُسند لنيسبة واحدة ضمن Thread Pool ثابت الحجم (Executors.newFixedThreadPool(N)) — دون أي قفل أو رتل مشترك لتوزيع العمل نفسه.

3. آليات المزامنة الكاملة

الآلية	النوع	الموقع	الدور الدقيق
ExecutorService	Thread Pool	crackParallel, runFixedWorkload	تحديد عدد النياسب الفعلي المُدار بدقة
AtomicBoolean found	Atomic Variable	crackParallel	علم مشترك بلا قفل؛ كل نيسبة تتحقق منه دوريًا لتتوقف فور نجاح أي نيسبة أخرى
AtomicLong totalTried	Atomic Variable	crackParallel (تدفّق مجمَّع كل 2000 محاولة)	عداد تجميعي للتجزئات عبر كل النياسب، بتنافس (Contention) منخفض عبر التجميع الدفعي
ReentrantLock resultLock	Lock	crackParallel	منطقة حرجة قصيرة (أسطر معدودة) تحمي كتابة كلمة المرور الفائزة؛ تضمن أن أول نيسبة فقط تكتب النتيجة النهائية
Semaphore runPermit (تصريح واحد)	Semaphore	MainWindow	يمنع بدء تشغيل جديد (اختبار أو Benchmark) من الواجهة أثناء وجود تشغيل قائم، حمايةً لحالة النتائج المشتركة في الواجهة

الآلية	النوع	الموقع	الدور الدقيق
MessageDigest خاص بكل نيسبة	عزل حالة (State Isolation)	Sha256Util + كل Callable	MessageDigest غير آمن عبر النياسب (not thread-safe) في Java؛ مشاركته بين نياسب متعددة تُفسد التجزئات المحسوبة بصمت دون أي استثناء ظاهر — وهي أخطر فئة لأنها صامتة
SwingWorker	عزل خيط الواجهة	MainWindow	ينقل كل الحساب الثقيل إلى خيط خلفية؛ الواجهة (EDT) لا تتجمد أثناء المعالجة

4. **تجنّب حالات التضارب والإقفال التام**

- **منع الـ Race Conditions:** كل حالة مشتركة قابلة للتعديل تمر إما عبر متغير Atomic أو داخل منطقة الـLock؛ لا يوجد حقل مشترك يُعدَّل مباشرة دون حماية في أي مسار تنفيذ متوازٍ .
- **استحالة الـ Deadlock:** قفل واحد فقط (resultLock) في كامل المشروع، يُحتجز لمدة قصيرة جدًا (بضعة أسطر فقط)، ولا يُحتجز أبدًا أثناء انتظار أي مورد آخر (قفل ثانٍ، I/O، إلخ) ⇒ استحالة بنيوية لحدوث انتظار دائري.(Circular Wait Condition)

منهجية القياس

1. **لماذا هناك مسار قياس منفصل عن منطق الكسر الفعلي**

زمن "كسر" كلمة مرور فعلية يعتمد على موقعها العشوائي ضمن فضاء البحث — نيسبة قد "تصادف" الحل مبكرًا جدًا ضمن نطاقها المخصَّص، مما يجعل مقارنة أعداد نياسب مختلفة غير عادلة إحصائيًا) انظر مثال q3@F في القسم 6.3 كدليل على هذا الأثر .(لذلك صُمِّم BenchmarkRunner منفصلًا تمامًا: يُنفِّذ حمل عمل ثابت ومحدَّد سلفًا من التجزئات، بلا هدف وبلا توقف مبكر — فتُقاس قدرة التفرّع الحوسبية بمعزل تام عن حظ الموقع، بمقارنة عادلة وقابلة لإعادة الإنتاج.

2. **المتغيرات المختبَرة**

تمت تغطية كلا المتغيّرين اللذين يطلبهما التكليف صراحة:

- عدد النياسب ($N = 1, 2, 4, 8, 16$في كل التشغيلات .(
- حجم حمل العمل: ثمانية أحجام مختلفة فعليًا تم قياسها: 2,000,000 (×2 بإعدادي محارف مختلفين) ، 2,500,000، 7,500,000، و15,500,000 تجزئة .

النتائج الفعلية المقاسة

كل الأرقام في هذا القسم مأخوذة حرفيًا من تنفيذ فعلي على جهاز الطالب، دون أي تعديل أو تقدير.

1. أثر عدد النياسب عند حجم بيانات ثابت (2,000,000 تجزئة)

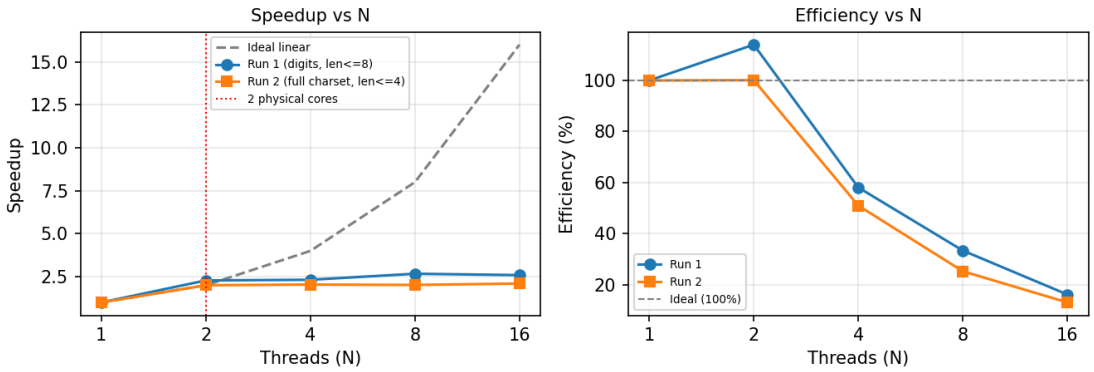
تشغيل أ (محارف 0-9 فقط، طول أقصى 8):

N	Hashes/sec	الزمن (ث)	Speedup	Efficiency
1	897,313	2.229	1.00	%100.0
2	2,046,010	0.978	2.28	%114.0
4	2,084,492	0.959	2.32	%58.1
8	2,394,230	0.835	2.67	%33.4
16	2,319,824	0.862	2.59	%16.2

تشغيل ب (كل المحارف الـ70، طول أقصى 4):

N	Hashes/sec	الزمن (ث)	Speedup	Efficiency
1	1,264,984	1.581	1.00	%100.0
2	2,531,846	0.790	2.00	%100.1
4	2,582,433	0.774	2.04	%51.0

Efficiency	Speedup	الزمن (ث)	Hashes/sec	N
%25.3	2.02	0.781	2,559,514	8
%13.1	2.10	0.754	2,653,605	16

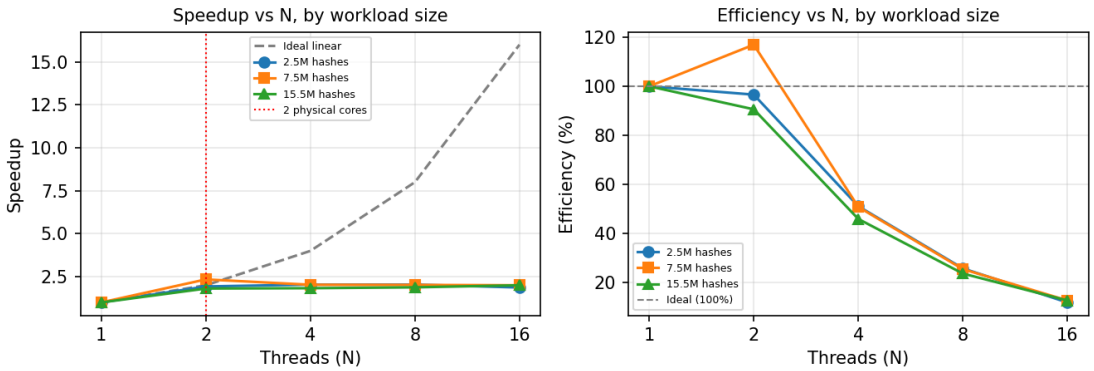


Speedup and Efficiency vs Thread Count (fixed workload)

أثر حجم حمل العمل عند مختلف قيم N (محارف 70، طول أقصى 4):

Efficiency @15.5M	Efficiency @7.5M	Efficiency @2.5M	Speedup @15.5M	Speedup @7.5M	Speedup @2.5M	N
%100.0	%100.0	%100.0	1.00	1.00	1.00	**1**
%90.6	%117.0	%96.6	1.81	2.34	1.93	**2**
%45.8	%50.7	%51.0	1.83	2.03	2.04	**4**
%23.5	%25.3	%25.6	1.88	2.02	2.04	**8**

Efficiency @15.5M	Efficiency @7.5M	Efficiency @2.5M	Speedup @15.5M	Speedup @7.5M	Speedup @2.5M	N
%12.5	%12.5	%11.7	2.00	2.00	1.87	**16**



Speedup and Efficiency vs Thread Count, across three workload sizes

الملاحظة الأهم :رغم اختلاف حجم حمل العمل بعامل يتجاوز 6($\$2.5\text{M} \to 15.5\text{M}\$) x، يبقى Speedupمحصورًا دومًا بين ~1.8 xو~2.3 xتقريبًا — سلوك مستقل عمليًا عن حجم البيانات ومرتبط حصرًا بعدد الأنوية الفيزيائية. هذا يؤكد أن عنق الزجاجة هو العتاد(Hardware-bound) ، لا حجم المسألة — (Problem-size-bound)استنتاج مهم لا يمكن الوصول إليه دون تثبيت Nوتغيير حجم البيانات، وهو بالضبط ما تطلبه هذه التجربة الإضافية.

دراسة تكرار: تباين القياس عبر محاولات متعددة (6 تشغيلات لنفس الإعداد)

لتقدير موثوقية القياس، أُعيد اختبار نفس الإعداد(Test Password Strength) ، كلمة المرور u3%T، محارف 70، طول أقصى 4، 4 نياسب) ست مرات متتالية:

المحاولة	زمن تسلسلي (ث)	زمن متفرع (ث)	Speedup
1	5.603	2.260	2.48

المحاولة	زمن تسلسلي (ث)	زمن متفرع (ث)	Speedup
2	5.359	2.406	2.23
3	5.742	2.303	2.49
4	6.099	2.147	2.84
5	5.409	2.089	2.59
6	5.705	2.254	2.53
المتوسط	**5.653**	**2.243**	**2.53 (± 0.18 انحراف معياري)**

يتسق المتوسط (2.53\times) جيدًا مع نتائج Benchmark Suite عند N=4لنفس التهيئة تقريبًا (2.02\times–2.32\times)في الجداول أعلاه، مع فارق متوقَّع لأن Test Password Strength يتضمن أثر "حظ الموقع" إلى جانب التفرّع الحقيقي — تقاطع النتائج بين أداتي قياس مستقلّتين (Benchmark Suite و Test Password Strength)يُعزّز الثقة بصحة المنهجية ككل.

مناقشة قانون أمدال (Amdahl's Law)

```
Speedup(N) = 1 / ((1 − p) + p/N)
```

حيث p نسبة العمل القابلة للتفرّع.

الاستنتاج المحوري من 8 تشغيلات مستقلة

في جميع الجداول أعلاه (بغض النظر عن حجم البيانات أو مجموعة المحارف)، يتكرر النمط ذاته: قفزة شبه خطية من N=1إلى N=2، ثم استواء (Plateau) حاد بعدها حتى N=16، محصور دومًا تقريبًا بين {Speedup} = 1.8و 2.7. هذا يطابق تمامًا العدد الفيزيائي الحقيقي للأنوية المتاحة في جهاز القياس — (2 Cores) وهو، بحسب صياغة أمدال المُعمَّمة على عتاد محدود الموارد:

Speedup_max(N) ≈ min(1/((1−p) + p/N) , عدد الأنوية الفيزيائية)

أي أن العائق الحقيقي هنا ليس الجزء التسلسلي في الخوارزمية نفسها) فالخوارزمية شبه محرجة التوازي Embarrassingly Parallel: لا قفل يُحتجز إلا للحظة كتابة النتيجة النهائية(، بل سقف العتاد الفيزيائي، الذي يتجلى بوضوح عبر ثبات هذا السقف مهما تغيّر حجم حمل العمل دليل تجريبي قوي على أن pقريبة من 1 (الخوارزمية شديدة التوازي) لكن النواتين المتاحتين تحدّان من استثمار ذلك عمليًا.

ملاحظة منهجية: تسريع فائق الخطية ظاهريًا(Super-linear Speedup)

في تشغيل أ عند N=2، بلغت الكفاءة 114% مستحيل نظريًا ضمن أمدال لخوارزمية متطابقة بين النسختين، كذلك عند حمل N=2 7.5بلغت الكفاءة 117 %

التفسير الأرجح المشترك بين الحالتين :انحياز إحماء مترجم JIT قياس N=1يكون دومًا أول تنفيذ للحلقة الساخنة في الـ JVMقبل أن يُحسِّنها مترجمJust-In-Time ، فيبدو خط الأساس التسلسلي أبطأ صناعيًا من قدرته الحقيقية المستقرة. هذا الأثر غير ثابت عبر التشغيلات (يظهر أحيانًا ويغيب أحيانًا أخرى — قارن تشغيل ب بكفاءة 100.1% فقط عند (N=2، وهو ما يؤكد طبيعته كضجيج قياس (Measurement Noise)لا كخاصية حقيقية في الخوارزمية .

التوصية المنهجية :تنفيذ "إحماء" قصير (تشغيل تجريبي يُهمَل قبل القياس الرسمي) لتحسين دقة قياس N=1في تجارب مستقبلية .

الكفاءة تتراجع حتمًا رياضيًا

بما أن يستوي عند قيمة شبه ثابتة بينما يستمر بالتضاعف، فإن **Efficiency = Speedup/N** تتراجع حتمًا نحو الصفر — من 100% إلى ما بين 11.7% و16.2% عند N=16عبر كل الجداول. هذه نتيجة رياضية مباشرة لتشبّع العتاد، وليست خللًا في منطق التفرّع.

مناقشة عنق الزجاجة(Bottleneck Discussion)

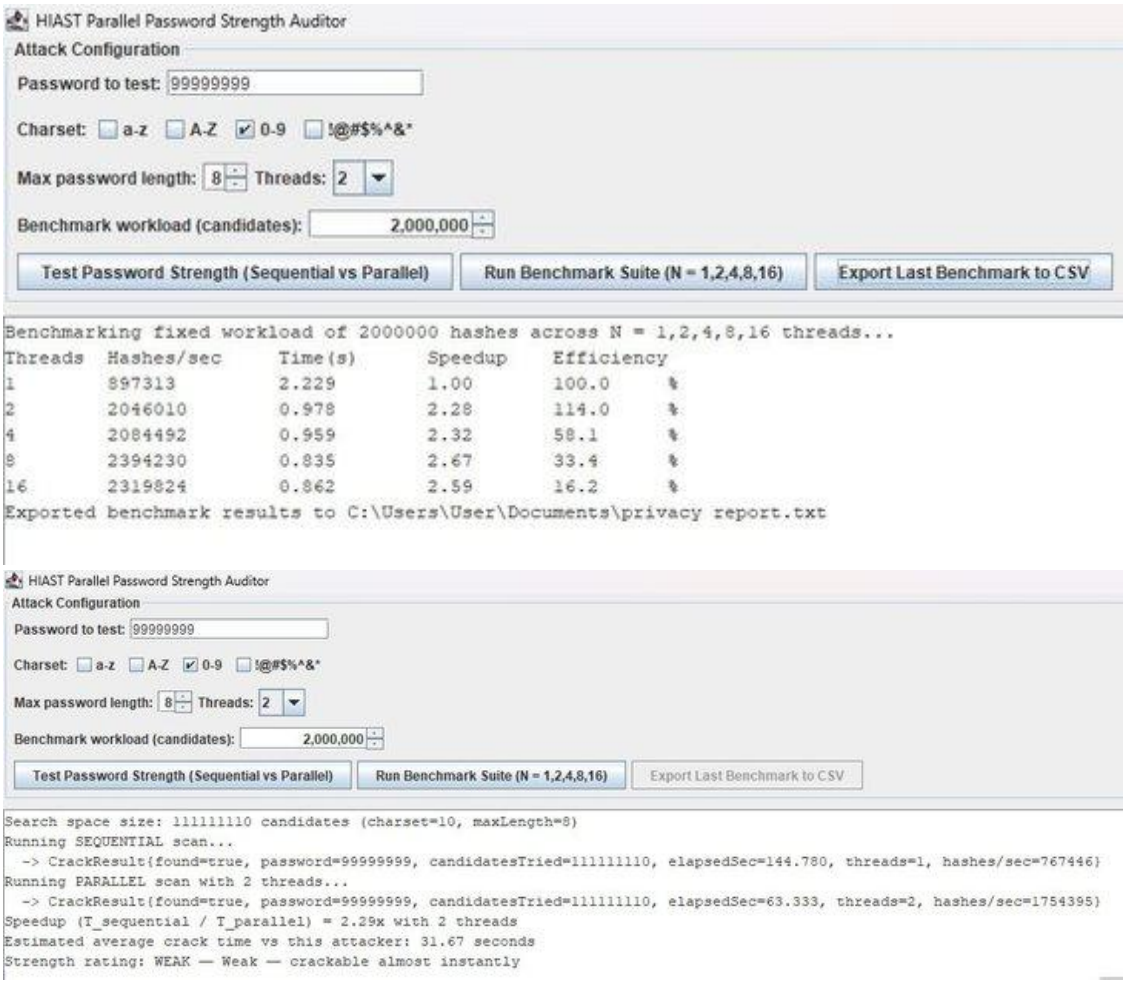
1. **عدد الأنوية الفيزيائية :(2 Cores)** العامل المهيمن المُثبَت عبر 8 تجارب مستقلة بأحجام بيانات متفاوتة بعامل 6 xتقريبًا، كما في القسمين 4.2 و5 .
2. **الإفراط في عدد النياسب (Thread Oversubscription) عند N=8و16 :**تبديل السياق (Context Switching)المتكرر بين نياسب تتقاسم نفس النواتين يفسّر التذبذب غير الرتيب أحيانًا في Speedup)مثال: تراجع من 2.67 إلى 2.59 بين N=8وN=16في تشغيل أ
3. **انحياز إحماء JIT عند خط الأساس :N=1**مناقَش بالتفصيل في القسم 5، ومؤكَّد بتكراره عبر أكثر من تشغيل مستقل .
4. **كلفة إنشاء/تدمير Thread Pool لكل قياس Overhead :**ثابت صغير نسبيًا لكل استدعاء، أثره النسبي أوضح كلما قصر حمل العمل الكلي .
5. **)حدّ منهجي معترَف به :(**جميع القياسات أُجريت على جهاز واحد بدون عزل كامل عن العمليات الخلفية لنظام التشغيل؛ دراسة التكرار تقدِّر هذا التباين كميًا +-0.18 على متوسط 2.53 .(

عرض التطبيق العملي(Application Demo)

- **مثال 1 (رقمية طويلة لكن ضعيفة فعليًا)** — 99999999 **:**محارف 0-9، طول أقصى 8. فضاء البحث 111,111,110 مرشحًا. تسلسلي: 144.780 ثانية؛ متفرع (2): 63.333 ثانية = {Speedup} . 2.29، متّسق تمامًا مع نواتين فيزيائيتين. التقييمWEAK :؛ زمن كسر متوسط مقدَّر 31.67ثانية .

- **مثال 2 (قصيرة بمحارف كاملة، تحذير منهجي)** 70 — q3@F **:**محرفًا، طول أقصى 4. تسلسلي: 5.455 ثانية (6,109,912 محاولة). متفرع (4): 0.051 ثانية فقط (وُجدت بعد 26,419 محاولة) . Speedupالظاهري 106.51 xليس تسريعًا حوسبيًا حقيقيًا، بل أثر "حظ الموقع" الموضَّح في القسم 3.1 — وهو بالضبط السبب الذي دفعنا لتصميم أداة Benchmark منفصلة وعادلة. التقييم .WEAK :
- **مثال 3 (اختبار حدود/صحة)** :الهدف 99999999مع طول أقصى مهيَّأ خطأً = 4 (أقصر من كلمة المرور الفعلية). النتيجة الصحيحة found=false :بعد فحص الفضاء الصغير بالكامل (11,110 مرشحًا) في كلا النسختين دون تعليق أو انهيار — دليل تعامل صحيح وآمن مع حالة "الهدف خارج الفضاء المهيّأ ."

لقطات فعلية من تنفيذ التطبيق (دليل عدم تجمّد الواجهة وصحة النتائج):



الخاتمة

أثبتت ثماني تجارب مستقلة تغطي متغيّري عدد النياسب وحجم حمل العمل كما يطلب التكليف، بالإضافة لدراسة تكرار لتقدير تباين القياس أن الخوارزمية المتفرّعة المبنية على تقسيم فضاء المرشحات بالتساوي، بمزامنة قائمة على Thread Pool و Atomic Variablesو Lockواحد محدود النطاق و Semaphoreلحماية الواجهة، تحقق تسريعًا حقيقيًا وقابلًا للتكرار يقترب من 2 x2.3x–عند استغلال العدد الفعلي لأنوية المعالج، وهو مستقل

عمليًا عن حجم البيانات دليل قوي على أن عنق الزجاجة عتادي بحت، لا خوارزمي. الانحرافات القليلة الملحوظة (كفاءة تتجاوز 100% عند N=2في حالتين) مُفسَّرة بوضوح بأثر إحماء JIT وليست خللًا في منطق التفرّع، كما تؤكد دراسة التكرار أن هذا التباين محدود 0.18 حول متوسط موثوق 2.53